Master Document 3: System Architecture, Tech Stack & API Integrations

Project Name: Agri-Grid: Smart Agriculture & Food Security Ecosystem
Project Lead: Ameer Muawiyah (Roll No: 1512)
Context: Technical Rulebook & Infrastructure Blueprint

1. Core Development Tech Stack

To ensure system stability and align with the team's engineering strengths, Agri-Grid employs a strictly defined technology stack. This stack separates the data processing engine from the web server and the user interface.

- **Backend Processing & Data Engine: Python** is the primary processing language. It is utilized heavily by the Lead Data Architect to write the ETL (Extract, Transform, Load) pipelines, interact with external APIs, and run the predictive mathematical models.
- **Web Server Framework:** A lightweight Python framework (such as **Flask** or **Django**) will be used by the Core Backend Engineer to build the secure routing logic and serve the data to the web browser.
- **Database:** A relational **SQL** database (e.g., MySQL, PostgreSQL, or SQLite3 for prototyping) is mandatory. The system requires complex JOIN operations, ACID-compliant financial transactions (for escrow and subsidies), and strict schema enforcement that NoSQL databases cannot efficiently provide for this specific use case.
- **Frontend UI:** Standard **HTML, CSS, and JavaScript**. The UI/UX Developer will use these to create dynamic, mobile-responsive interfaces (crucial for the Transporter routing and Farmer field inputs).

2. Engineering & Planning Toolchain

The team adheres to industry-standard tools to manage the Agile-Incremental software development lifecycle (SDLC) and parallel V-Model testing.

- **System Modeling: StarUML** and **Draw.io** for blueprinting System Architecture, Use Cases, Sequence, and Class diagrams.
- **Documentation: Overleaf / MS Word** for drafting the formal Software Requirements Specifications (SRS).
- **Project Management: Trello / Jira** for Agile sprint tracking, task delegation, and backlog management.
- **Version Control: Git / GitHub** for centralized code management. The QA/DevOps Lead enforces branch protection rules to prevent direct pushes to the main branch without peer review.
- **Automated Testing: JUnit** (or PyTest for Python) for verifying backend mathematical logic, and **Selenium** for validating frontend UI interactions.

3. External API Integrations

Agri-Grid does not rely on static dummy data for its environmental variables. It integrates with real-world public APIs to pull hyper-local telemetry, transforming the application into a "Smart" ecosystem.

- **The Trigger Mechanism:** The farmer inputs their exact location via dropping a pin or entering a zip code. The frontend converts this into precise Latitude and Longitude GPS coordinates.
- **Weather Telemetry:** The backend passes these coordinates to standard weather APIs (like **OpenWeatherMap** or **Tomorrow.io**) to retrieve historical rainfall data, current humidity, and 14-day forecasts.
- **Soil & Agronomic Telemetry:** The backend utilizes specialized agricultural APIs (like **Agromonitoring** or **SoilGrids**) to retrieve satellite-driven data, including soil moisture percentages, soil temperature at a 10cm depth, and historical vegetation health indexes.

4. Data Architecture & Algorithmic Logic

The external APIs provide raw JSON data, but they do not possess farming logic. The system's intelligence resides entirely within the Python backend, engineered by the Lead Data Architect.

- **The Prediction Engine:** The Python scripts ingest the JSON payloads from the Weather/Soil APIs and combine them with the farmer's manual inputs (e.g., Crop Type, Planting Date) stored in the SQL database.
- **Execution:** Custom algorithms process these combined datasets (e.g., *"If Crop == Wheat AND Soil Moisture < 20% AND No Rain Forecasted for 3 Days ➔ Output: Alert farmer to irrigate immediately"*). This logic powers the Predictive Yield Forecaster, the Agronomist's Diagnostic Workbench, and the Compliance Verification Engine.

5. System Optimization & Resilience Strategies

To ensure the prototype functions reliably without crashing or hitting commercial paywalls, the architecture incorporates two advanced engineering optimizations:

5.1. API Database Caching (Rate-Limit Protection)

Public APIs operate on "freemium" models with strict rate limits (e.g., max 1,000 calls per day). To prevent server blocks, Agri-Grid employs a caching strategy.

- **The Logic:** Instead of pinging the OpenWeatherMap API every single time a farmer refreshes their dashboard, the Python backend fetches the weather data once, saves (caches) it directly into the Agri-Grid SQL database, and timestamps it.
- **The Result:** For the next 3 to 6 hours, any dashboard refreshes will instantly read the weather data from the local SQL database rather than exhausting the external API limits.

5.2. Offline Flash Caching (Logistics Resilience)

The Transporter module's Cold-Chain Telemetry Logger must function in rural agricultural zones with poor 4G/5G cellular coverage.

- **The Logic:** The frontend UI is designed to utilize HTML5 Local Storage (Flash Caching). If the delivery truck loses network connection, the application continues to log temperature telemetry locally on the driver's mobile device browser.

- **The Result:** The moment the device detects a restored cellular connection, the web app automatically bulk-syncs the cached temperature payload to the central Python backend, ensuring the B2B Buyer has a continuous, unbroken temperature graph.